Procesos de Desarrollo de Software

CLASE 9 – MAPEOS DE OBJETOS

Agenda de la clase

- ORM
- JPA
- Hibernate
- Magic Café evolución

Problema de JDBC

- Queremos guardar objetos Java en una base de datos (por ejemplo “Producto”)
- MySQL NO entiende objetos Java.
- MySQL entiende:
  - tablas
  - columnas
  - filas
  - SQL

```java
public class Producto {

    private Long id;
    private String nombre;
    private double precio;
}
```

Traducción a SQL

```
PreparedStatement stmt =
        connection.prepareStatement(
                "SELECT * FROM productos"
        );

ResultSet rs = stmt.executeQuery();

List<Producto> productos = new ArrayList<>();

while(rs.next()) {
    Producto producto = new Producto();

    producto.setId(rs.getLong("id"));
    producto.setNombre(rs.getString("nombre"));
    producto.setPrecio(rs.getDouble("precio"));

    productos.add(producto);
}
```

```
Producto producto = new Producto("Cafe Latte", 4500);

Connection connection = DriverManager.getConnection(URL, USER, PASSWORD);

PreparedStatement stmt =
        connection.prepareStatement(
                "INSERT INTO productos(nombre, precio) VALUES (?, ?)"
        );

stmt.setString(1, producto.getNombre());
stmt.setDouble(2, producto.getPrecio());

stmt.executeUpdate();
```

- ¿Qué ocurre si Producto cambia?
  - **private** String descripcion;
- ¿Qué ocurre con las relaciones?

Relaciones con JDBC

```
while(rs.next()) {

    Pedido pedido = new Pedido();
    pedido.setId(rs.getLong("pedido_id"));

    ItemPedido item = new ItemPedido();
    item.setCantidad(rs.getInt("cantidad"));

    Producto producto = new Producto();
    producto.setNombre(rs.getString("producto_nombre"));

    item.setProducto(producto);
    pedido.getItems().add(item);
}
```

- Estamos reconstruyendo manualmente el grafo de objetos

Queremos que una herramienta haga automáticamente el mapeo entre objetos y tablas.

ORM

- **O**bject **R**elational **M**apping
- **Mapear objetos ↔ Tablas relacionales.**
- Traduce automáticamente objetos a SQL y viceversa.
- Ejemplo:

```
Producto producto = new Producto("Cafe Latte", 4500);
entityManager.persist(producto);
```

- ORM genera internamente:

```
INSERT INTO productos(nombre, precio)
VALUES ('Cafe Latte', 4500);
```

ORM

- Un ORM evita:
  - PreparedStatement
  - ResultSet
  - Mapping manual
  - Setters repetitivos
  - SQL repetitivo

ORMs

Lenguaje	ORM
Java	Hibernate
C#	Entity Framework
Python	SQLAlchemy
Node.Js	Sequelize / TypeORM
PHP	Doctrina

JPA + Hibernate

- JPA (**J**ava **P**ersistence **A**PI): es la especificación
- Hibernate: es la implementación ORM

JPA (**J**ava **P**esistence **A**PI)

- Es una **especificación estándar de Java** para trabajar con persistencia de objetos.
- JPA define una forma estándar de guardar, buscar, actualizar y eliminar objetos Java en una base de datos relacional.
- JPA no es una librería concreta. Es un conjunto de reglas, interfaces y anotaciones. Ejemplos:
  - @Entity
  - @Id
  - @OneToMany
  - @ManyToOne

Anotaciones

- **¿Qué es una anotación?**
- Son “metadatos” para darle información especial a Java o frameworks.
  - Las anotaciones son etiquetas que los frameworks interpretan.
- Ejemplo:

```
@Override
public String toString() {

}
```

- Java entiende que este método se sobreescribe.
- La etiqueta no hace ”magia” sola, es metadata para Java.

Conceptos principales de JPA

- Entity
- Id
- Relaciones
- EntityManager
- JPQL
- Persistence Unit

@Entity

- Marca una clase como persistible.
- JPA interpreta:
  - Clase **Cliente** ↔ Tabla **cliente**

```java
import jakarta.persistence.Entity;

@Entity
public class Cliente {

    private Long id;
    private String nombre;
}
```

- import jakarta.persistence.Entity; solo trae la definición de la anotación (metadata para Hibernate).
- Hibernate/JPA: esta clase debe persistirse en la base de datos.

@Table

- Permite especificar el nombre real de tabla.

```java
import jakarta.persistence.Entity;

@Entity
@Table(name = "clientes")
public class Cliente {

    private Long id;
    private String nombre;
}
```

@Id

- Marca la clave primaria.
- Equivalente SQL conceptual

```
id BIGINT PRIMARY KEY
```

- **@GeneratedValue**
  - Hace que la DB genere IDs automáticamente.

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
```

```
import jakarta.persistence.Entity;

@Entity
@Table(name = "clientes")
public class Cliente {

    @Id
    private Long id;
    private String nombre;
}
```

@GenerateValue

- Al usar:

```
@Id
@GeneratedValue(strategy = ...)
private Long id;
```

- La estamos indicando a JPA/Hibernate cómo generar la clave primaria

Estrategia	Cómo genera IDs	En SQL
IDENTITY	la DB genera el ID automáticamente	CREATE TABLE productos (id BIGINT AUTO_INCREMENT PRIMARY KEY);
SEQUENCE	usa secuencias	CREATE SEQUENCE producto_seq; SELECT nextval('producto_seq');
TABLE	usa una tabla auxiliar	id_generator **tabla** | **siguiente_id** producto | 5
AUTO	Hibernate decide	Hibernate elige: IDENTITY, SEQUENCE o TABLE. Según DB, dialect, provider

@Column

- Configura columnas.

```java
import jakarta.persistence.Entity;

@Entity
@Table(name = "clientes")
public class Cliente {

    @Id
    private Long id;
    @Column(nullable = false, length = 100)
    private String nombre;
}
```

Relaciones en JPA

- JPA permite mapear relaciones entre objetos.

Relación	Significado
@OneToOne	Uno a uno
@OneToMany	Uno a muchos
@ManyToOne	Muchos a uno
@ManyToMany	Muchos a muchos

@OneToOne

- **Uno a Uno**
- **UNA** persona tiene **UN** pasaporte.

- Conceptualmente genera:
- persona
- └── pasaporte_id

```java
@Entity
public class Persona {

    @Id
    private Long id;

    private String nombre;

    @OneToOne
    private Pasaporte pasaporte;
}
```

@OneToMany

- **Uno a Muchos**
- **1 Cliente → muchos Pedidos**

```java
@Entity
public class Cliente {

    @Id
    private Long id;

    private String nombre;

    @OneToMany(mappedBy = "cliente")
    private List<Pedido> pedidos = new ArrayList<>();
}
```

- mappedBy = "cliente"
- Significa que la relación se controla desde el atributo:
- ***private Cliente*** *cliente;* que existe en Pedido.

@ManyToOne

- **Muchos a uno**
- **Muchos** pedidos pertenecen a **un** cliente.
- Conceptualmente genera:
- pedido
- └── cliente_id

```java
@Entity
public class Pedido {

    @Id
    private Long id;

    @ManyToOne
    @JoinColumn(name = "cliente_id")
    private Cliente cliente;
}
```

NOTA: **@OneToMany** y **@ManyToOne** suelen ir juntos!

Cliente **1** ——— ***** Pedido

@JoinColumn

- Define la FK.
- Genera conceptualmente:
  - cliente_id

```java
@ManyToOne
@JoinColumn(name = "cliente_id")
private Cliente cliente;
```

@ManyToMany

- **Muchos a muchos**
- **Muchos** alumnos pueden estar en **muchos** cursos.

```java
@Entity
public class Alumno {

    @Id
    private Long id;

    @ManyToMany
    private List<Curso> cursos = new ArrayList<>();
}
```

- Esto genera una tabla intermedia:
  - alumno_curso

Cascade

- Es muy importante. Sirve para propagar operaciones entre entidades relacionadas.

```
@OneToMany(cascade = CascadeType.ALL)
```

- Si guardo el padre:
  - también se guardan los hijos.
- Ejemplo:

```
entityManager.persist(cliente);
```

- Automáticamente guarda el cliente, sus pedidos y las relaciones.

Sin Cascade

```java
@OneToMany(mappedBy = "pedido")
private List<ItemPedido> items;
```

```java
Pedido pedido = new Pedido();

ItemPedido item1 = new ItemPedido();
ItemPedido item2 = new ItemPedido();

pedido.getItems().add(item1);
pedido.getItems().add(item2);

// Sin cascade, es necesario persistir cada entidad individualmente
entityManager.persist(item1);
entityManager.persist(item2);
entityManager.persist(pedido);
```

Con Cascade

```java
@OneToMany(mappedBy = "pedido", cascade = CascadeType.ALL)
private List<ItemPedido> items;
```

```java
Pedido pedido = new Pedido();

ItemPedido item1 = new ItemPedido();
ItemPedido item2 = new ItemPedido();

pedido.getItems().add(item1);
pedido.getItems().add(item2);

// Con CascadeType.ALL, los items se guardarán automáticamente al guardar el pedido
entityManager.persist(pedido);
```

Tipos de Cascade

Tipo	Propaga	Ejemplo
CascadeType.**PERSIST**	persist()	entityManager.persist(pedido);
CascadeType.**REMOVE**	remove()	entityManager.remove(pedido);
CascadeType.**MERGE**	merge()	entityManager.merge(pedido);
CascadeType. **ALL**	todos	Todos
CascadeType.**REFRESH**	refresh()	entityManager.refresh(pedido);
CascadeType.**DETACH**	detach()	entityManager.detach(pedido);

NOTA: Cascade tiene sentido cuando las entidades hijas dependen del ciclo de vida de la entidad padre (composición).

EntityManager

- Es el objeto principal de JPA.

Método	Acción
persist	guardar
find	buscar
merge	actualizar
remove	eliminar
createQuery	consultar

Ejemplos

```
// Guardar
entityManager.persist(cliente);

// Buscar
Cliente cliente = entityManager.find(Cliente.class, 1L);

// Eliminar
entityManager.remove(cliente);

// Actualizar
entityManager.merge(cliente);
```

JPQL

- **J**ava **P**ersistence **Q**uery Language
- **NO es SQL**
- Trabaja sobre:
  - entidades
  - atributos
- Diferencias: `FROM Cliente` <> `FROM clientes`

```
List<Cliente> clientes =
        entityManager
            .createQuery("FROM Cliente", Cliente.class)
            .getResultList();
```

EntityManagerFactory

- Su trabajo es **crear EntityManagers**

```
EntityManagerFactory emf =
        Persistence.createEntityManagerFactory("miPU");
```

- Hibernate:
  - lee persistence.xml
  - analiza entidades @Entity
  - construye metadata
  - configura dialect
  - configura conexiones
  - configura mapeos
  - prepara proxies
  - prepara cachés
  - prepara internamente **MUCHAS** cosas. Es costoso porque inicializa **TODO** Hibernate.

Flujo correcto

- Una única Factory (inicializamos una única vez)
- Muchos EntityManager
  - Representan una unidad de trabajo.

```
EntityManagerFactory emf =
        Persistence.createEntityManagerFactory("miPU");

EntityManager em1 = emf.createEntityManager();
EntityManager em2 = emf.createEntityManager();
EntityManager em3 = emf.createEntityManager();
```

- Normalmente hacemos Singleton con **EntityManagerFactory**

Persistence Unit

- Se configura en: **META-INF**/persistence.xml
- **¿Qué contiene?**
  - conexión DB
  - entidades
  - provider
  - propiedades Hibernate
- Ejemplo mínimo:

```
<persistence-unit name="miPU">

    <provider>
        org.hibernate.jpa.HibernatePersistenceProvider
    </provider>

</persistence-unit>
```

Hibernate Dialect

- El dialect le dice a Hibernate:
- "¿Qué tipo de base de datos estoy usando y cómo habla SQL esa base?"
- Ejemplo:

```
<property name="hibernate.dialect"
          value="org.hibernate.dialect.MySQLDialect"/>
```

- Indica a Hibernate que estoy usando MySQL.
- Hibernate adapta:
  - CREATE TABLE
  - IDs
  - LIMIT
  - joins
  - tipos
  - funciones SQL

Magic Cafe

- **Objetivo**: de JDBC manual a persistir objetos Java en MySQL, manteniendo Swing + MVC + DAO.

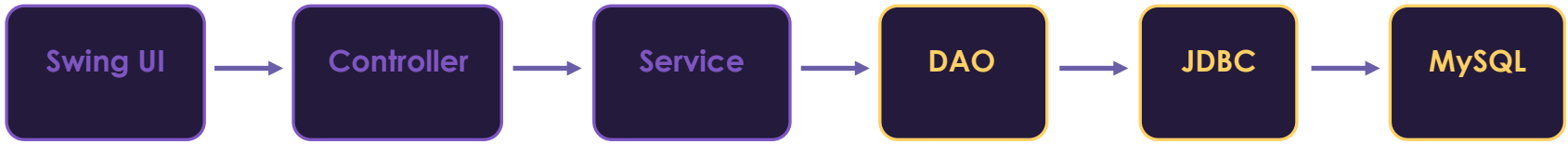
Evolución de la clase

JDBC puro
↓
DAO + Singleton
↓
DAO + JPA/Hibernate

La UI de escritorio se mantiene.

Punto de Partida

- Tenemos una aplicación de escritorio que guarda pedidos en MySQL.
- **Problema**: el DAO conoce SQL, tablas, columnas, PreparedStatement, ResultSet y transacciones manuales.



Usando ORM


Objeto Java
Pedido
Producto
ItemPedido
ORM
JPA
Hibernate
Base de datos
pedidos
productos
pedido_items

Cambio principal en MagicCafe

- No tiramos la UI ni MVC: reemplazamos la tecnología dentro del DAO.

ANTES



AHORA



Dependencias Maven

- Necesitamos Hibernate, JPA y el driver de MySQL.
  - hibernate-core: implementación ORM.
  - jakarta.persistence-api: anotaciones JPA.
  - mysql-connector-j: driver JDBC de MySQL.
  - La app sigue siendo Java Desktop/Swing.

```
<dependency>

<groupId>org.hibernate.orm</group
Id>
  <artifactId>hibernate-
core</artifactId>
</dependency>

<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-
j</artifactId>
</dependency>
```

Persistence.xml

- Archivo de configuración JPA. Define conexión, entidades y comportamiento de Hibernate.

```
<class>com.uade.swingmagiccafe.model.Producto</clas
s>

<class>com.uade.swingmagiccafe.model.Pedido</class>

<class>com.uade.swingmagiccafe.model.ItemPedido</cl
ass>

  <property name="jakarta.persistence.jdbc.url"

value="jdbc:mysql://localhost:3306/magiccafe_jpa"
/>
```

Qué explica esto

- Unidad de persistencia
- Proveedor Hibernate
- Clases @Entity
- Conexión MySQL
- Creación/actualización de tablas

Mapeo de Producto

- Una clase Java se convierte en una tabla.

```
@Entity
@Table(name = "productos")
public class Producto {

    @Id
    @GeneratedValue(strategy =
GenerationType.IDENTITY)
    private int id;

    @Column(nullable = false, length = 120)
    private String nombre;

    @Column(nullable = false)
    private double precio;
}
```

@Entity

La clase pasa a ser persistible.

@Table

Define el nombre de la tabla.

@Id + @GeneratedValue

Clave primaria generada por MySQL.

Mapeo de Pedido

- Pedido tiene datos propios y una colección de items.

```
@Entity
@Table(name = "pedidos")
public class Pedido {

    @Id
    @GeneratedValue(strategy =
GenerationType.IDENTITY)
    private int id;

    private double subtotal;
    private String descuentoAplicado;
    private double total;
    private LocalDateTime fechaCreacion;

    @OneToMany(mappedBy = "pedido",
               cascade = CascadeType.ALL,
               orphanRemoval = true)
    private List<ItemPedido> items = new
ArrayList<>();
}
```

OrphanRemoval

Elimina hijos que quedan desconectados del padre.

Mapeo de ItemPedido

- ItemPedido conecta Pedido con Producto.

```
@Entity
@Table(name = "pedido_items")
public class ItemPedido {

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "pedido_id", nullable =
false)
    private Pedido pedido;

    @ManyToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "producto_id", nullable =
false)
    private Producto producto;

    private int cantidad;
    private double precioUnitario;
    private double subtotal;
}
```

Fetch

- Indica cuándo Hibernate debe cargar relaciones.

Pedido
└── List<ItemPedido>
ItemPedido
└── Producto

- Al hacer:

```
ItemPedido item = entityManager.find(ItemPedido.class, 1L);
```

- ¿Hibernate debe traer automáticamente:
  - Pedido?
  - Producto?
  - ambas?
  - ninguna?

Fetch

Tipo	Significado
EAGER	cargar inmediatamente
LAZY	cargar solo cuando se necesite

```
@ManyToOne(fetch = FetchType.EAGER)
private Producto producto;
```

Fetch Eager

- Cuando Hibernate carga ItemPedido
  - También carga automáticamente Producto

```
@ManyToOne(fetch = FetchType.EAGER)
private Producto producto;
```

```
ItemPedido item = entityManager.find(ItemPedido.class, 1L);
```

- Hibernate trae:
  - ItemPedido
  - Producto

Fetch Lazy

- Traelo solo si hace falta
- Cuando Hibernate carga ItemPedido
  - **NO** trae Pedido todavía.
  - **Hibernate trae un "proxy"**

```
@ManyToOne(fetch = FetchType.LAZY)
private Pedido pedido;
```

```
item.getPedido(); // Recién acá ejecuta la query
```

JPAUtil como Singleton

- **EntityManagerFactory** es costoso: se crea una vez y se reutiliza.

```
public class JPAUtil {

    private static JPAUtil instance;
    private EntityManagerFactory emf;

    private JPAUtil() {
        emf =
Persistence.createEntityManagerFactory("magicca
fePU");
    }

    public static JPAUtil getInstance() {
        if (instance == null) instance = new
JPAUtil();
        return instance;
    }

    public EntityManager crearEntityManager() {
        return emf.createEntityManager();
    }
}
```

DAO con EntityManager

- El DAO sigue encapsulando persistencia, pero ya no escribe SQL manual.

```
public class PedidoDAO {

    public PedidoConfirmadoDTO guardar(Pedido pedido,
...) {
        EntityManager em =
JPAUtil.getInstance().crearEntityManager();
        try {
            em.getTransaction().begin();

            pedido.confirmar(nombreDescuento,
subtotal, total);
            em.persist(pedido);

            em.getTransaction().commit();
            return new PedidoConfirmadoDTO(...);
        } catch (RuntimeException e) {
            if (em.getTransaction().isActive())
em.getTransaction().rollback();
            throw e;
        } finally {
            em.close();
        }
    }
}
```

persist()

Guarda el objeto como fila.

transaction

commit o rollback igual que antes.

close()

Cierra el EntityManager.

Consultas: de SQL a JPQL

- JPQL consulta entidades, no tablas directamente
  - SQL habla de tablas y columnas.
  - JPQL habla de clases y atributos.
  - Hibernate se encarga de generar el SQL real.

```
// Antes: SQL
SELECT * FROM productos ORDER BY nombre;

// Ahora: JPQL
SELECT p FROM Producto p ORDER BY
p.nombre
```

```
List<Producto> productos = entityManager
    .createQuery("SELECT p FROM Producto p ORDER BY
p.nombre", Producto.class)
    .getResultList();
```

Transacciones JPA

- Seguimos necesitando transacciones cuando una operación debe guardarse completa

```
try {
    em.persist(pedido);

em.getTransaction().commit();
} catch (RuntimeException e) {
    if
(em.getTransaction().isActive(
)) {

em.getTransaction().rollback()
;
    }
    throw e;
}
```

Recordatorio

Un pedido y sus items deben guardarse juntos.

Si falla un item, no queremos un pedido incompleto en la base.

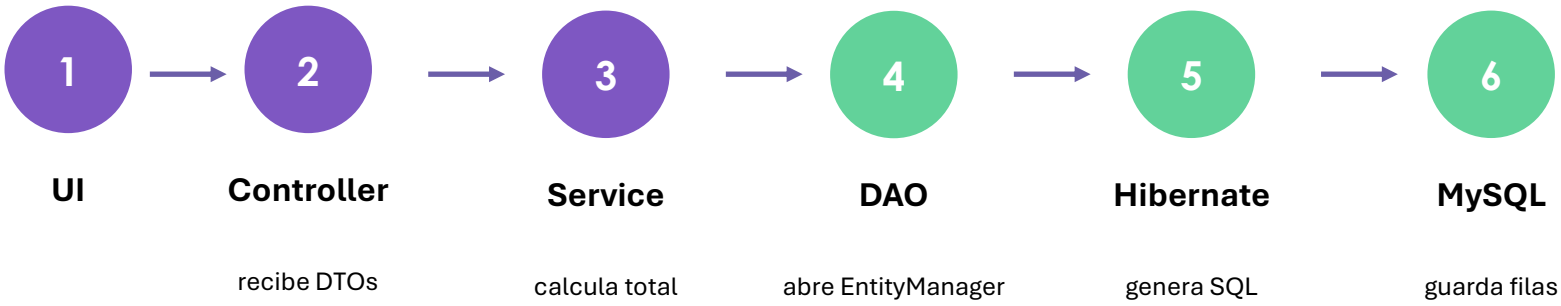
Cascade

- La relación @OneToMany puede propagar operaciones
- **Cascade** evita guardar manualmente cada ItemPedido.

```
@OneToMany(mappedBy = "pedido",
           cascade = CascadeType.ALL,
           orphanRemoval = true)
private List<ItemPedido> items = new
ArrayList<>();
```

em.persist(pedido) → Hibernate detecta items → INSERT pedidos + items

Flujo Completo



Errores frecuentes

Entidad sin constructor vacío

JPA necesita constructor vacío o protected.

Relación sin FK

Falta @JoinColumn o mappedBy incorrecto.

Tabla vieja

hibernate update no borra columnas antiguas.

Transacción faltante

persist() requiere transacción activa.

Level up!

- Agregar una entidad **Cliente**.
- Relacionar **Pedido** con **Cliente** usando @**ManyToOne**.
- Listar pedidos por cliente usando **JPQL**.
- Agregar un método en **PedidoDAO**: listarPorCliente(idCliente).
- Mostrar en la UI el nombre del cliente en la tabla de pedidos.

Gracias!

- Preguntas?
- mailto: mstricagnoli@uade.edu.ar

